

Name of the Student	V Yogeswar
Reg. No	192425175
GitHub Link	https://github.com/Yogeswar2906/MLA0405---DEEP-LEARNING/tree/main/Assessment/CO3

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2

Game Based Learning

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0405

SLOT : D

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Duration: 1 Hour

Assessment Tool Weightage: 35%

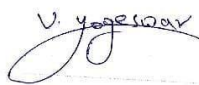
Marks: 50

Code of Conduct:

I V Yogeswar/ 192425175 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student.



Name of the student: V Yogeswar

Criteria	Understanding of Concept (2.5)	Conceptual Clarity (2.5)	Problem-Solving Ability (2)	Solution Design & Application (2)	Teamwork & Game Participation (1)	Total (10)
Weightage	25 %	25%	20%	20 %	10 %	100%
Round 1						
Round 2						
Total						

Signature of the Faculty:

Total Marks :

Question:

Design and participate in a two-round game-based learning activity to explore the fundamental concepts of Convolutional Neural Networks (CNNs), including architecture, layers, filters, parameter sharing, regularization, AlexNet, ResNet, and real-world applications. Progress through five levels in each round, applying your knowledge to solve challenges involving CNN design, feature extraction, architecture comparison, and practical applications.

The Game Progression

Round 1:

Basic CNN Concepts → Filters → Parameter Sharing → Architecture → AlexNet vs ResNet

Round 2:

Convolution → Feature Maps → Regularization → Architecture Comparison → Real-World Application

ROUND 1: CNN Architecture Builder (Learn by constructing and understanding CNN architectures layer by layer)

Level	Challenge	Description
Level 1	CNN Layer Matcher	Match CNN layers such as Convolution, Pooling, Flatten, and Fully Connected layers with their respective functions and roles in the network.
Level 2	Filter Finder	Identify suitable filters and kernels for feature extraction and understand how filter size, stride, and padding affect the output feature map.
Level 3	Parameter Sharing Puzzle	Determine how the same filter weights are reused across different image locations and calculate the number of parameters in a convolutional layer.
Level 4	CNN Architecture Debugger	Identify and correct problems in a CNN architecture, such as incorrect layer order, incompatible dimensions, and inappropriate layer selection.
Level 5	AlexNet vs ResNet Challenge	Compare AlexNet and ResNet architectures and identify how their architectural approaches differ, particularly the use of residual connections in ResNet.

ROUND 2: CNN Feature Detective (Learn by exploring how CNNs extract features, avoid overfitting, and solve real-world problems)

Level	Challenge	Description
Level 1	Edge Detection Explorer	Apply simple (3\times3) filters to an image and determine how convolution operations can detect edges and other basic features.
Level 2	Feature Map Detective	Analyze feature maps produced by different CNN layers and identify the progression from simple features such as edges to complex patterns and objects.

Level	Challenge	Description
Level 3	Regularization Rescuer	Identify overfitting in a CNN model and select suitable regularization techniques to improve generalization and model performance.
Level 4	CNN Architecture Race	Compare ResNet and AlexNet for different image-classification applications and determine which architecture is more suitable for a given problem.
Level 5	CNN Application Master	Design a CNN-based solution for a real-world application such as medical image analysis, face recognition, object detection, or autonomous driving, and justify the selected architecture and layers.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Conceptual Clarity	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Problem-Solving Ability	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design & Application	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Teamwork & Game Participation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

CNN GAME-BASED LEARNING ACTIVITY

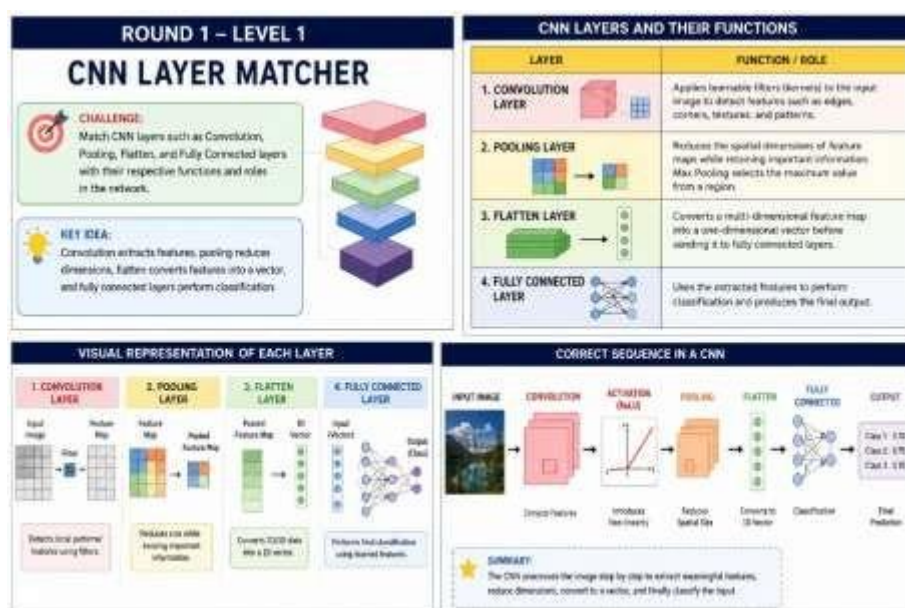
ROUND 1: CNN Architecture Builder

Level 1 – CNN Layer Matcher

CNN Layer	Function / Role
Convolution Layer	Applies learnable filters/kernels to the input image to detect features such as edges, corners, textures, and patterns.
Pooling Layer	Reduces the spatial dimensions of feature maps while retaining important information. Max Pooling selects the maximum value from a region.
Flatten Layer	Converts a multi-dimensional feature map into a one-dimensional vector before sending it to fully connected layers.
Fully Connected Layer	Uses the extracted features to perform classification and produces the final output.

Correct sequence:

Input Image → Convolution → Activation (ReLU) → Pooling → Flatten → Fully Connected → Output



Key idea: Convolution extracts features, pooling reduces dimensions, flatten converts features into a vector, and fully connected layers perform classification.

Level 2 – Filter Finder

A CNN uses small kernels/filters to detect different features in an image.

For example, a 3×3 **edge-detection filter** can be:

This filter detects **horizontal edges**.

Another filter:

can detect **vertical edges**.

Effect of filter parameters

- **Filter size:** Determines the region examined at each convolution step.
- **Stride:** Determines how far the filter moves at each step.
- **Padding:** Adds pixels around the image to control the output size.
- **Number of filters:** Determines the number of feature maps generated.

The output size is:

where:

- = input size
- = filter size
- = padding
- = stride

Example

For a image, filter, stride , and padding :

Therefore, the output feature map is:

Level 3 – Parameter Sharing Puzzle

CNNs use **parameter sharing**, meaning that the same filter weights are reused at different positions of an image.

For example, consider a convolution layer with:

- Filter size =
- Input channels = 3
- Number of filters = 32
- One bias per filter

Number of parameters:

Therefore, the convolution layer contains **896 trainable parameters**.

Why parameter sharing is important

Parameter sharing:

1. Reduces the number of parameters.
2. Reduces computational requirements.
3. Helps detect the same feature at different image locations.
4. Improves CNN efficiency.

Example: A filter that detects an edge in the top-left corner can also detect the same type of edge in the center or bottom-right region.

Level 4 – CNN Architecture Debugger

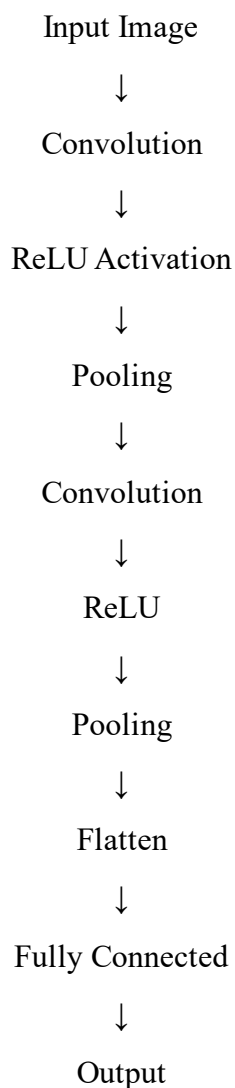
Consider the following incorrect architecture:

Input → Flatten → Convolution → Pooling → Fully Connected

Problems

1. **Flatten is placed too early.**
 - Flatten converts the feature map into a 1D vector.
 - A convolution layer expects spatial information.
2. **Convolution should come before Flatten.**
 - CNNs need spatial information to extract image features.
3. **Pooling should normally follow convolution/activation.**

Correct architecture



Debugging conclusion

The major problem was the incorrect placement of the Flatten layer. The CNN should first extract spatial features using convolution and pooling and only then flatten the feature maps for classification.

Level 5 – AlexNet vs ResNet Challenge

Feature	AlexNet	ResNet
Introduced	2012	2015
Main idea	Deep CNN with convolution and pooling	Deep CNN using residual learning
Architecture	Conventional sequential architecture	Uses residual/skip connections
Activation	ReLU	ReLU
Important contribution	Demonstrated strong GPU-based deep learning performance	Solved degradation problems in very deep networks
Depth	8 learned layers	Can have 18, 34, 50, 101, 152 layers, etc.
Skip connections	No	Yes
Training very deep networks	Difficult	Easier
Main advantage	Historically important and comparatively simple	Effective for very deep networks

Residual connection

ResNet introduces a shortcut connection:

Instead of learning the complete transformation directly, the network learns a **residual function** .

Conclusion

AlexNet was a major breakthrough in deep CNNs, while **ResNet** introduced residual connections that made training much deeper networks practical.

ROUND 2: CNN Feature Detective

Level 1 – Edge Detection Explorer

Consider the following input:

Use the horizontal edge filter:

Element-wise multiplication and summation:

The large magnitude indicates a strong **horizontal edge**.

Interpretation

CNN filters detect basic visual patterns such as:

- Horizontal edges
- Vertical edges

- Diagonal edges
- Corners
- Textures

These basic features are combined by deeper layers to identify complex objects.

ROUND 2 – LEVEL 1

EDGE DETECTION EXPLORER

Apply simple (3x3) filters to an image and determine how convolution operations can detect edges and other basic features.

CHALLENGE

Use 3x3 filters on an image patch and observe how convolution helps detect edges and basic features.

LEARNING GOAL

Understand how CNN convolution with small filters can detect edges, lines, corners and other basic patterns.

Input Image (Patch)

3x3 Filter (Kernel)

Convolution (Output)

Convolution slides the filter across the image and computes the element-wise multiplication and sum to produce the output.

HOW CONVOLUTION DETECTS EDGES

• A filter (kernel) is a small matrix of weights. When it is convolved with an image, it highlights specific patterns.

Original Image (Grayscale)

Edge Filter (3x3)
Horizontal Edge Detector

Feature Map (Output)

Explanation:
The filter has negative values on top and positive values at bottom. When placed over an area with a horizontal intensity change, the sum becomes large (positive or negative), highlighting the edge in the output feature map.

EXAMPLE: CONVOLUTION CALCULATION

Input Patch (3x3)

Filter (Kernel)

Element-wise Multiplication

Sum of all elements = $-10 \cdot -10 - 10 \cdot 0 + 0 \cdot 0 + 0 \cdot 0 + 0 \cdot 0 + 0 \cdot 0 = -30$

Output = -30 → Large magnitude indicates the presence of a horizontal edge.

Interpretation

- Large positive or negative values in the output mean strong edge or feature.
- Values close to 0 mean no significant change (flat region).

OTHER BASIC 3x3 FILTERS

Vertical Edge Detector

Diagonal Edge Detector (↖)

Sharpening Filter

Output Detects Vertical Edges

Output Detects Diagonal Edges

Output Enhances Details

Different filters capture different basic features. CNNs learn many such filters automatically during training.

WHAT WE LEARNED

1. Convolution with small filters (like 3x3) can detect basic features such as edges and lines.
2. Different filters detect different orientations and patterns.
3. These basic features in early layers help CNNs learn complex patterns in deeper layers.

REAL-WORLD APPLICATIONS

Image Enhancement

Medical Image Analysis

Autonomous Driving

Face Recognition

KEY TAKEAWAY
CNNs start by detecting simple features like edges using small filters. These are the building blocks for recognizing complex objects!

Level 2 – Feature Map Detective

CNN layers progressively learn more complex features.

Early layers

Detect:

- Edges
- Lines
- Corners
- Simple textures

Middle layers

Detect:

- Shapes
- Curves
- Object parts
- Patterns

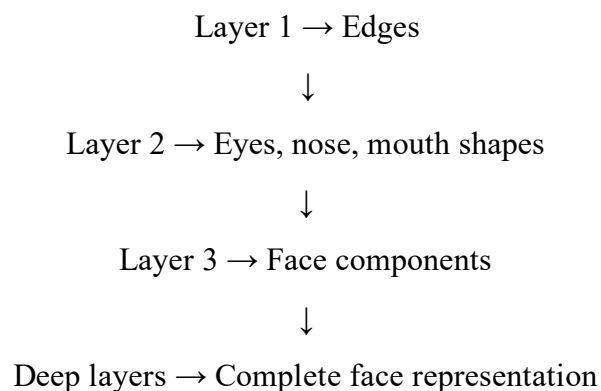
Deep layers

Detect:

- Faces
- Animals
- Vehicles
- Objects
- Complex structures

Example

For a face-recognition CNN:



Therefore, CNN feature extraction progresses from **simple features to complex semantic features**.

Level 3 – Regularization Rescuer

Problem

Suppose a CNN produces:

- Training accuracy = **98%**
- Validation accuracy = **75%**

This indicates **overfitting**.

The model performs extremely well on training data but poorly on unseen data.

Suitable solutions

1. Dropout

Randomly disables some neurons during training.

Example:

Dropout = 0.5

Approximately 50% of selected neurons are randomly ignored during a training step.

2. Data Augmentation

Creates additional variations of training images through:

- Rotation
- Flipping
- Cropping
- Scaling
- Brightness changes

3. L2 Regularization

Adds a penalty for large weights to the loss function:

4. Early Stopping

Training is stopped when validation performance stops improving.

Recommended solution

For an image classification problem, a combination of:

Data Augmentation + Dropout + Early Stopping

can effectively reduce overfitting.

Level 4 – CNN Architecture Race

Suppose two applications are given:

Application A: Small image classification dataset

Recommended architecture: AlexNet-style CNN

Reason:

- Relatively simpler architecture.
- Suitable for learning basic image features.
- Easier to train compared with very deep networks.

Application B: Large-scale complex image classification

Recommended architecture: ResNet

Reason:

- Supports very deep networks.
- Residual connections improve gradient flow.
- Can learn complex hierarchical features.
- Generally more suitable for challenging image classification tasks.

Comparison

Requirement	AlexNet	ResNet
Simple architecture	✓	
Very deep network		✓

Residual/skip connections		✓
Complex feature learning	Good	Excellent
Deep network training	More difficult	Easier
Modern image classification	Less preferred	More preferred

Winner

For most modern complex image-classification applications:
is the better choice.

Level 5 – CNN Application Master

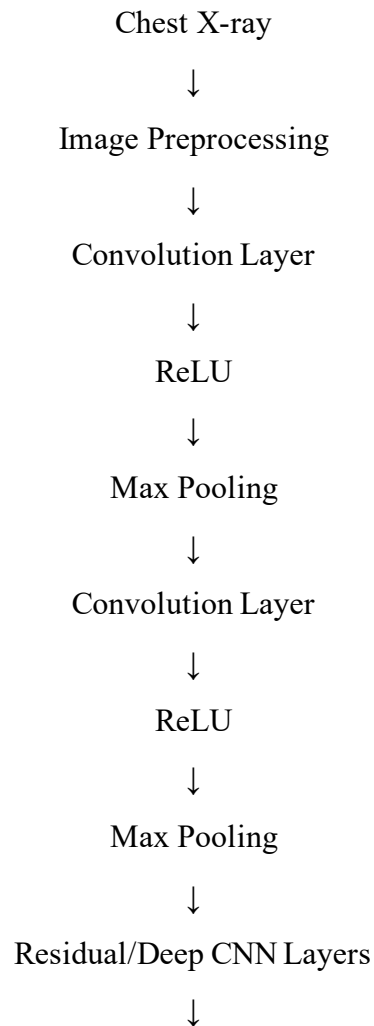
Application: Medical X-Ray Image Classification

Problem

Develop a CNN that classifies chest X-ray images into:

- **Normal**
- **Disease affected**

Proposed architecture



Global Average Pooling



Fully Connected Layer



Sigmoid Output



Normal / Disease

Why ResNet?

ResNet is suitable because:

1. It can learn complex visual patterns.
2. Residual connections allow deeper networks to be trained effectively.
3. It can extract hierarchical features from X-ray images.
4. Transfer learning can be used when the medical dataset is relatively small.

Output

For binary classification:

If:

the image can be classified as **Disease**.

Otherwise:

the image is classified as **Normal**.

Real-world applications of CNNs

CNNs can be applied to:

-  Medical image analysis
-  Face recognition
-  Autonomous driving
-  Traffic sign recognition
-  Object detection
-  Satellite image analysis
-  Industrial defect detection

Final Game Learning Summary

Round	Level	Main Concept	Key Learning
Round1	1	CNN Layers	Understand the role of each layer

	2	Filters	Learn feature extraction using kernels
	3	Parameter Sharing	Calculate and understand shared weights
	4	Architecture	Debug incorrect CNN designs
	5	AlexNet vs ResNet	Understand conventional vs residual architectures
Round2	1	Convolution	Detect edges using filters
	2	Feature Maps	Understand simple-to-complex feature extraction
	3	Regularization	Prevent overfitting
	4	Architecture Comparison	Select suitable CNN architectures
	5	Real-World Application	Design a CNN solution for practical problems

Overall conclusion

The two-round game demonstrates the complete CNN learning process, beginning with basic layers and filters and progressing toward architecture design, parameter calculation, feature extraction, regularization, architecture comparison, and real-world applications. Through the activities, learners understand why CNNs are highly effective for image-based tasks and why architectures such as ResNet are important for deep learning.

Presentation Pic

